

UNIDAD 1:

INTRODUCCION

CURSO

IA



Docente:
Gabriel Mosso



BRAINSTORM

Rep.Argentina 786, Sunchales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383



Que es la inteligencia artificial?

La **inteligencia artificial (IA)** es un campo de la informática que se centra en la creación de sistemas capaces de realizar tareas que normalmente requieren inteligencia humana. Esto incluye habilidades como el aprendizaje, la resolución de problemas, el reconocimiento de patrones, la toma de decisiones, la comprensión del lenguaje natural y la percepción visual. La IA se divide en dos categorías principales:

IA débil o IA estrecha: Está diseñada para realizar tareas específicas, como los asistentes virtuales (por ejemplo, Siri o Alexa), los algoritmos de recomendación en plataformas de streaming, o los sistemas de detección de fraude en las transacciones financieras. No posee conciencia ni entendimiento general más allá de la tarea para la que fue programada.

IA fuerte o IA general: Es una forma de IA que tiene la capacidad de entender, aprender y aplicar conocimientos de manera general, similar a la inteligencia humana. Este tipo de IA aún es teórico y no ha sido logrado.

1. IA Débil / Estrecha



Asistente



Algoritmo de
Recomendaciones



Fraud
Detección



Sistemas específicos. Ejemplos:
Asistentes (Siri/Alexa),
Recomendaciones, Detección de
Fraude. Sin conciencia general.

2. IA Fuerte / General



Teórico: Capacidad general para
entender, aprender y aplicar
conocimientos como humanos.
Aún no logrado.



1. El Entorno de Trabajo

1. El Entorno de Trabajo: Python



Orquestación
de flujo



Automatización
de datos

Rol: Infraestructura y lenguaje base.

Función en el pipeline:

- * Orquesta y conecta todo el flujo de trabajo.
- * Automatiza la carga de datos y el control de herramientas.
- * Sirve de interfaz para programar los algoritmos.

2. La Materia Prima: MNIST



Volumen de
imágenes
(dígitos)

2	8	3	5
2	4	3	1
1	4	0	7
4	9	5	6
3	3	5	8



División
de datos

Rol: Set de datos (dataset) de entrada.

Función en el pipeline:

- * Aporta el volumen de imágenes (dígitos manuscritos) para el proyecto.
- * Se divide en bloques de entrenamiento (para que la IA aprenda) y prueba (para evaluar su éxito).

3. El Motor de Procesamiento: TensorFlow



Diseño de
Arquitectura



Exportación
de Modelo



Rol: Framework de aprendizaje profundo (deep learning).

Función en el pipeline:

- * Diseña y construye la arquitectura de la red neuronal.
- * Entrena el algoritmo procesando los datos de MNIST dentro del entorno Python.
- * Exporta el modelo final optimizado y listo para predecir.



Estructura de una Red Neuronal

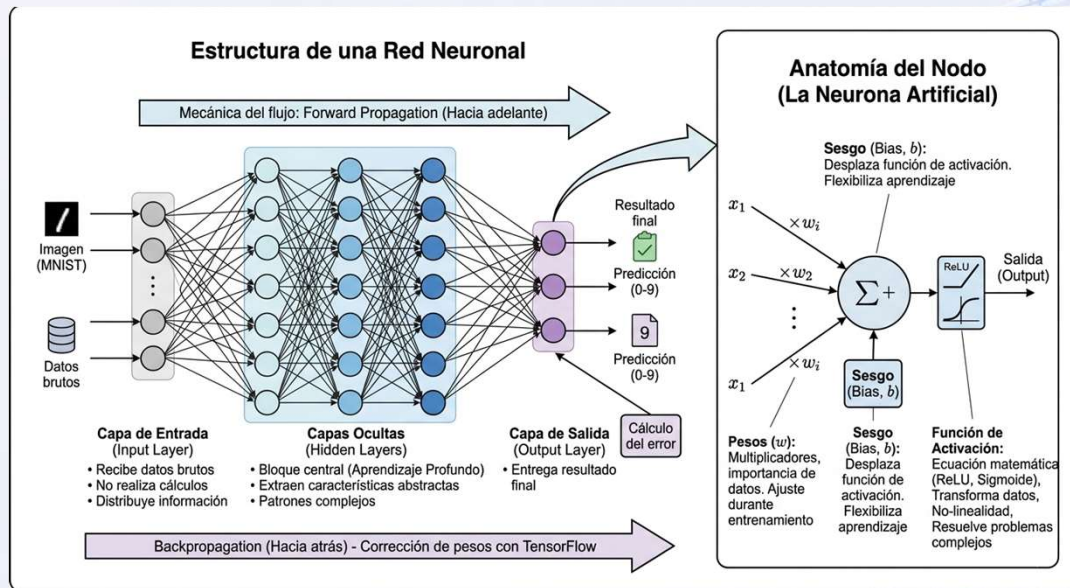
- **Capa de Entrada (Input Layer):** Recibe los datos brutos del exterior (ej. los píxeles de una imagen de MNIST). No realiza cálculos, solo distribuye la información.
- **Capas Ocultas (Hidden Layers):** Bloque central donde ocurre el aprendizaje profundo (*Deep Learning*). Pueden ser una o cientos de capas que extraen características abstractas y patrones complejos.
- **Capa de Salida (Output Layer):** Entrega el resultado final del procesamiento (ej. la predicción de qué número es, del 0 al 9).

Anatomía del Nodo (La Neurona Artificial)

Cada neurona dentro de las capas procesa la información en tres pasos matemáticos:

- **Pesos:** Multiplicadores que determinan la importancia de cada dato de entrada. Se ajustan durante el entrenamiento.
- **Sesgo (Bias, \$b\$):** Un valor que permite desplazar la función de activación para flexibilizar el aprendizaje.
- **Función de Activación:** Ecuación matemática (como *ReLU* o *Sigmoide*) que transforma la suma de los datos e introduce no-linealidad, permitiendo a la red resolver problemas complejos.

Mecánica del flujo: Los datos viajan hacia adelante (*Forward Propagation*), la red calcula el error y este se propaga hacia atrás (*Backpropagation*) usando **TensorFlow** para corregir los pesos y mejorar la precisión.





¿Qué significa entrenar una red neuronal?

El Concepto Central

Entrenar es el proceso de **ajustar automáticamente los parámetros internos** (pesos y sesgos) de la red para que sus predicciones sean lo más exactas posibles.

La analogía de la Caja Negra

- **La Entrada y Salida:** Introduces una pregunta (datos) y la red te devuelve una respuesta (predicción).
- **Las Perillas Internas:** Dentro de la caja hay miles o millones de perillas móviles (pesos y sesgos).
- **El Ajuste por Error:** Si la respuesta es incorrecta, un algoritmo mide el fallo y gira sutilmente las perillas para corregir el rumbo.
- **La Repetición:** Este ciclo de prueba y error se repite millones de veces con miles de ejemplos distintos.

El Objetivo Final del Entrenamiento

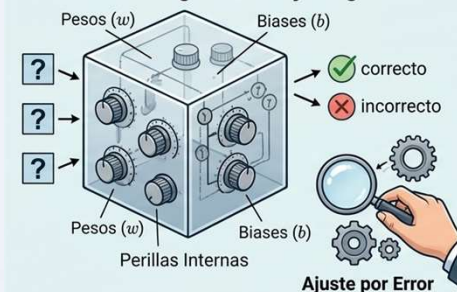
- **Calibración de precisión:** Encontrar la combinación y configuración óptima de todas las perillas internas (pesos y sesgos).
- **Éxito con datos conocidos:** Lograr que la red procese los datos de entrenamiento actuales y devuelva respuestas correctas.
- **Capacidad de generalización:** Conseguir que la red aplique lo aprendido y responda con éxito ante datos nuevos y desconocidos que jamás ha visto.

El proceso en una frase: Entrenar es el ciclo automatizado de calibrar pesos y sesgos para transformar datos de entrada en predicciones correctas.

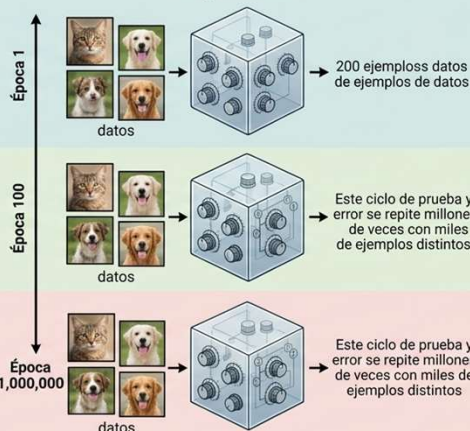
ENTRENAMIENTO DE UNA RED NEURONAL: CONCEPTOS CLAVE Y PROCESO

Entrenamiento de una red neuronal es conceptos clave y procedimiento automatizado que optimizan parámetros en el control de computadoras que realizan funciones de una red neuronal.

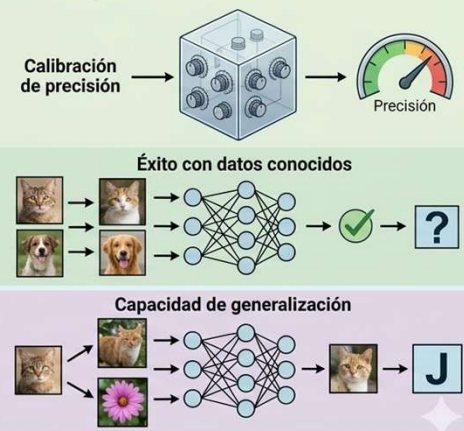
La analogía de la Caja Negra



Repetición



El Objetivo Final del Entrenamiento



Summariz: anuna una red neuronal: conceptos clave y proceso



El Proceso de Entrenamiento en 5 Pasos

1. Inicialización: Los pesos y biases de la red arrancan con valores aleatorios o configurados por defecto. En este punto, la red solo adivina al azar.

2. Propagación hacia adelante (*Forward Propagation*): Los datos de entrada viajan por las capas de la red. Cada neurona procesa la información y la pasa a la siguiente hasta generar una predicción final en la capa de salida.

3. Cálculo del Error (Función de Pérdida): El sistema compara la predicción de la red con el resultado real esperado y calcula matemáticamente qué tan grande fue la equivocación.

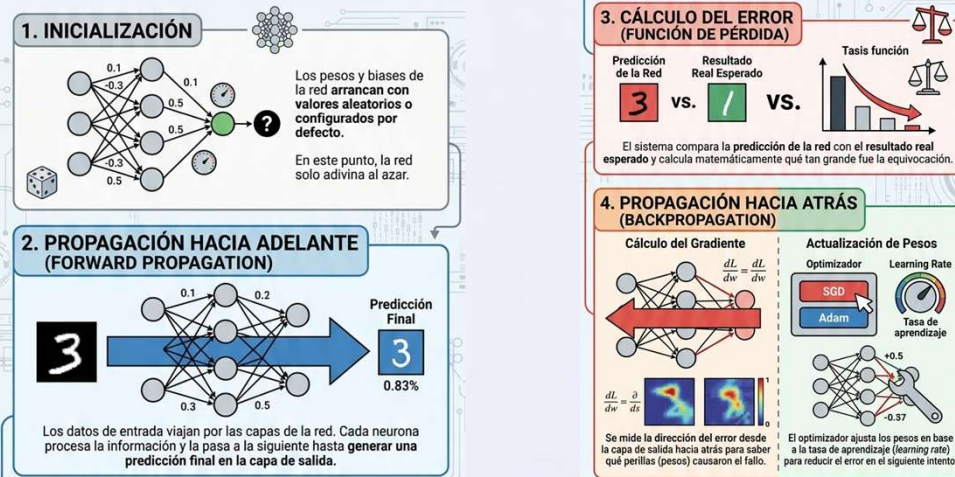
4. Propagación hacia atrás (*Backpropagation*): El paso clave para el aprendizaje, dividido en dos momentos:

Cálculo del Gradiente: Se mide la dirección del error desde la capa de salida hacia atrás para saber qué perillas (pesos) causaron el fallo.

Actualización de Pesos: El optimizador ajusta los pesos en base a la tasa de aprendizaje (*learning rate*) para reducir el error en el siguiente intento.

5. Repetición (Épocas): Todo el ciclo se repite miles de veces sobre el conjunto de datos. La red recalcula y ajusta sus parámetros continuamente hasta que el error es mínimo.

EL PROCESO DE ENTRENAMIENTO DE UNA RED NEURONAL EN 5 PASOS





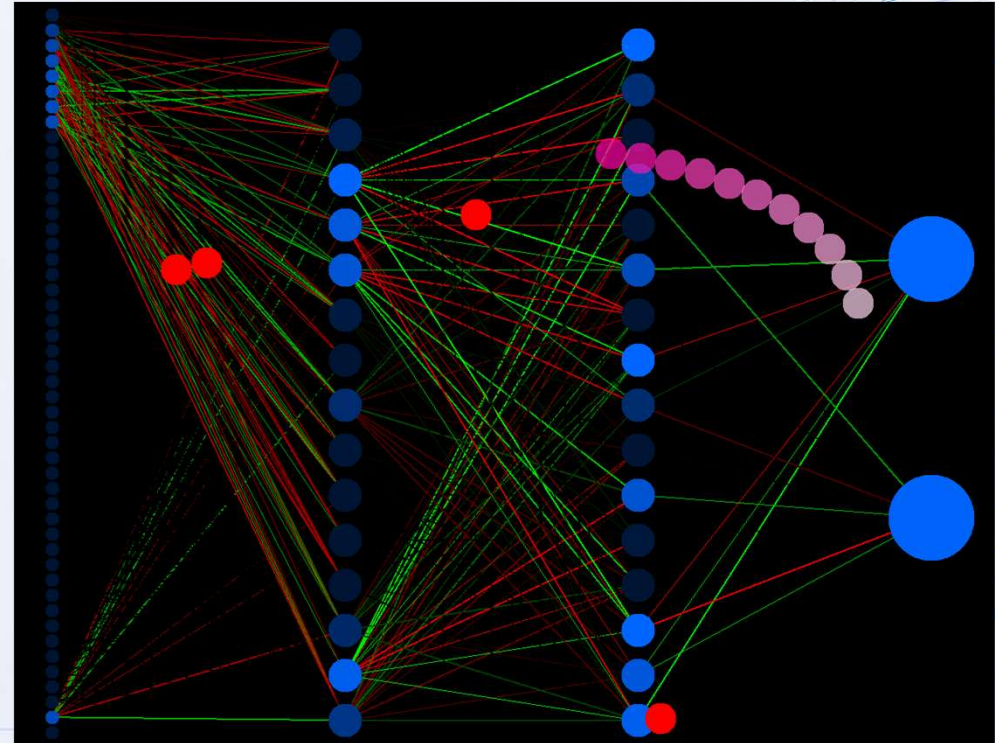
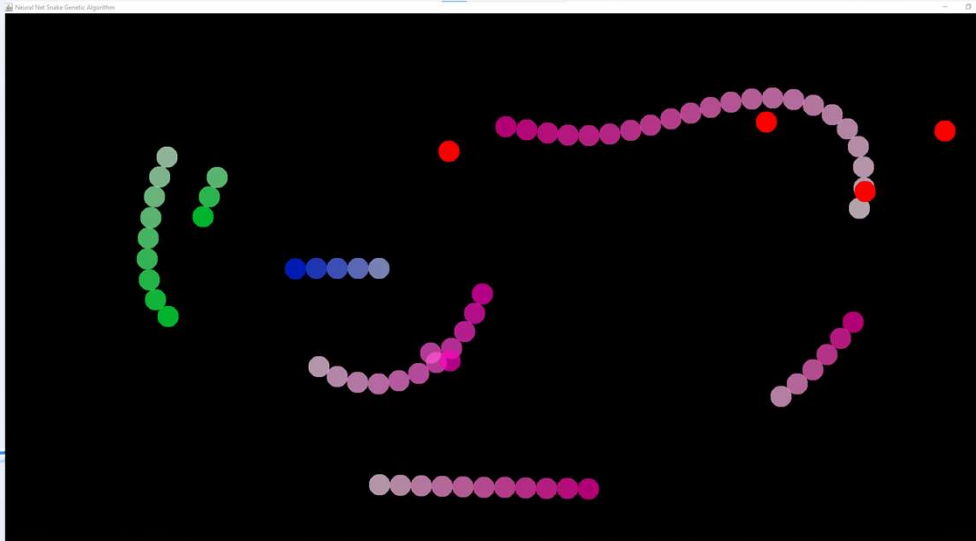
Proceso de entrenamiento Red Neuronal (SNAKE):

El SNAKE es un Gimnasio de red neuronal que pone el siguiente desafío a la IA:
Cada NN tiene:

- Un triángulo de visión de 128 Entradas.
- 2 «Hidden layers»
- 2 Salidas (Izquierda / Derecha)

Las SNAKE deberán:

- Evitar el borde.
- Comer (Puntos Rojos)
- Evitar sus propios cuerpos.





Demis Hassabis :

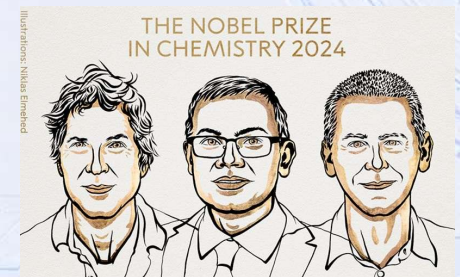
2016: AlphaGo, desarrollado por DeepMind bajo la dirección de Hassabis, derrotó al campeón mundial de Go, Lee Sedol, en una serie histórica que demostró el potencial de la inteligencia artificial en juegos complejos.

2018: Hassabis fue elegido miembro de la Royal Society (FRS) y nombrado asesor de la Oficina de Inteligencia Artificial del Reino Unido. Ese mismo año, DeepMind anunció avances significativos en la predicción de estructuras de proteínas con AlphaFold.

2020: AlphaFold 2 logró un progreso notable al predecir estructuras proteicas con una precisión comparable a métodos experimentales, resolviendo un desafío de décadas en la biología.

2021: DeepMind publicó las estructuras de más de 200 millones de proteínas, facilitando el trabajo de investigadores en todo el mundo.

2024: Hassabis, junto con su colega John Jumper, fue galardonado con el Premio Nobel de Química por el desarrollo de AlphaFold, una herramienta de inteligencia artificial que predice la estructura de proteínas. Este avance es crucial para resolver problemas científicos complejos y desarrollar tratamientos para enfermedades como el cáncer y el alzhéimer.

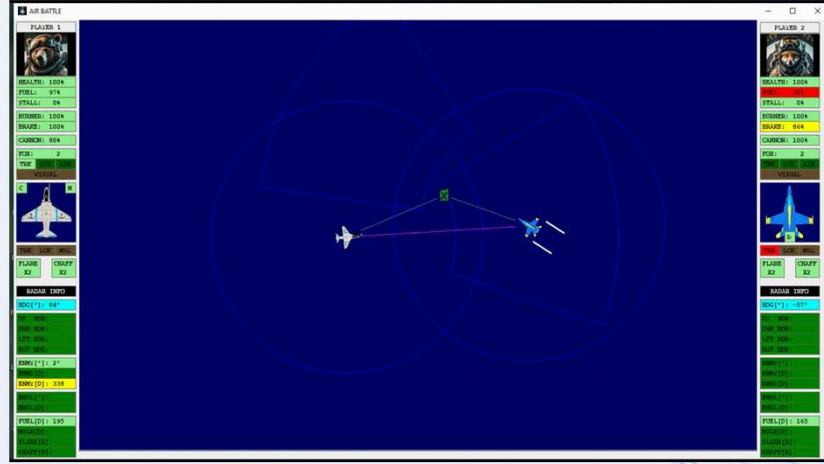
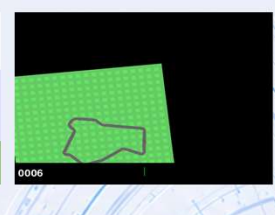
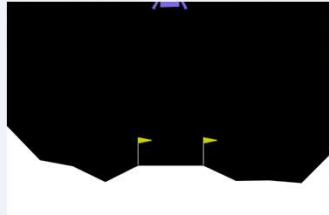


	Proteínas descubiertas	Años
Metodos Tradicionales	170.000	74
Alpha fold	200.000.000	1



HERRAMIENTAS DIDACTICAS IA

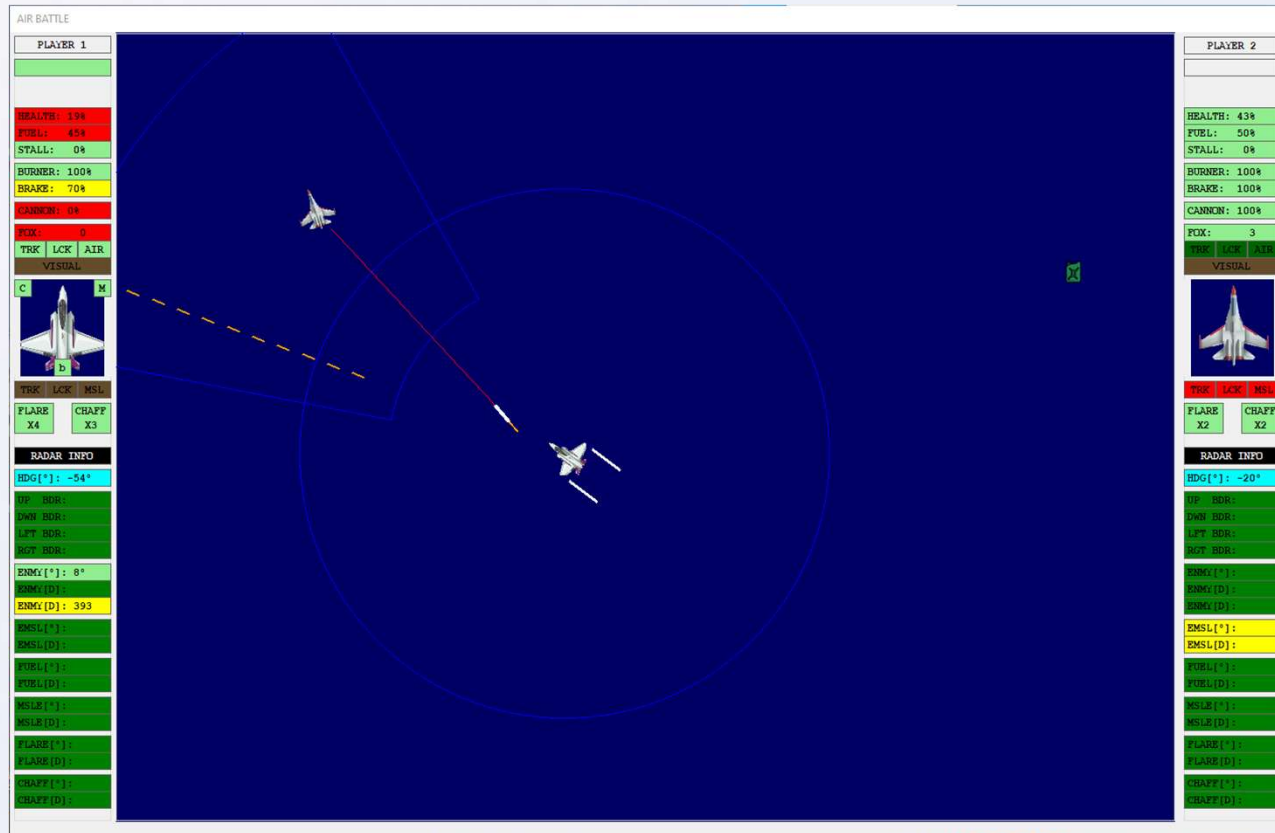
Nombre	Plataforma	Tipo de entornos	Ventajas destacadas	URL oficial / Repositorio
Gymnasium (ex OpenAI Gym)	Python	Juegos simples, control clásico, Atari	Modular, ligero, comunidad activa, fácil de integrar con algoritmos estándar	https://gymnasium.farama.org
DeepMind Control Suite	Python + MuJoCo	Control motor, robótica	Física realista, útil para benchmarking	https://github.com/deepmind/dm_control
Unity ML-Agents	Unity + Python	Juegos 3D, simulaciones complejas	Gráficos 3D, multijugador, integración con Deep Learning	https://unity.com/products/machine-learning-agents
PyBullet	Python	Simulación robótica	Gratuito, física precisa, alternativa libre a MuJoCo	https://pybullet.org/ / https://github.com/bulletphysics/bullet3
MuJoCo	Python	Simulación avanzada de robots	Alta fidelidad, usado en investigación avanzada	https://mujoco.org
PettingZoo	Python	Multiagente (juegos y simulaciones)	Diseñado para múltiples agentes, integración con Gym y Stable-Baselines	https://www.pettingzoo.ml
Meta-World	Python + MuJoCo	Manipulación robótica multitarea	Reutilización de conocimiento entre tareas, entorno realista de manipulación	https://github.com/rlworkgroup/metaworld
MineRL	Python + Java	Minecraft	Ambiente naturalista, tareas jerárquicas, integración con aprendizaje profundo	https://minerl.io/ / https://github.com/minerllabs/minerl
BrainStorm Air Battle	Python + Java	Juego 2D Combate	Desarrollo de estrategias, supervivencia.	



El AIR BATTLE es una herramienta didáctica desarrollada por BRAINSTORM, que permite entrenar inteligencia artificial y le permite al alumno entender los procesos y dificultades propias del proceso de desarrollo de IA a través de la librería TENSORFLOW.

El objetivo del curso es Entrenar una red neuronal para que comande el avión de manera exitosa con el objetivo de:

- Atacar al enemigo cuando lo detecta en su campo visual.
- Evadir el ataque enemigo
- Evite colisionar con el borde del mapa.
- Recolecte los items del mapa.





```
01 import openai
02 from IPython import
03     ChatOpenAI
04
05 def generar_respuesta(prompt):
06     modelo = ChatOpenAI(
07         model = "gpt-4o"
08         temperature = 0.7
09     )
10     respuesta = modelo.invoke(
11     ) prompt
12     return respuesta.content
13
14
15 if __name__ == "__main__":
16     print("¡Gracias!")
17     print("dudas, consultas...")
18     input("Presione una tecla para salir...")
```

> print("Gracias !")

> print("dudas, consultas...")

> input("Presione una tecla para salir...")



Docente:
Gabriel Mosso



BRAINSTORM

Rep. Argentina 786, Sunchoales, Sta fe.



GabrielNMosso@GMail.com



Tel: +54 351 3733383

